Conception logicielle et design patterns

Informatique — Licence 3 (L3)

Cours INF-L3-04

Institut Africain de Management (IAM) — Bamako, Mali
Bibliothèque Universitaire — Bronbah Tech

À propos de ce cours

Après la programmation orientée objet (L2) et la modélisation UML (L2), ce cours apprend à concevoir un logiciel qui reste agréable à faire évoluer : les cinq principes SOLID, les patrons de création, structurels et comportementaux les plus utiles, les architectures en couches et hexagonale, puis le refactoring comme hygiène quotidienne contre la dette technique.

Document pédagogique original créé pour les étudiants de l'IAM : leçons détaillées, exemples commentés, travaux dirigés et pratiques, exercices corrigés, QCM d'évaluation et vidéos recommandées. Aucun contenu n'est reproduit d'ouvrages existants.

Sommaire

• Introduction à la matière
• Objectifs d'apprentissage
• Leçon 1 — Principes de conception : le sigle SOLID expliqué par l'exemple
• Leçon 2 — Patterns de création : fabrique et singleton
• Leçon 3 — Patterns structurels : adaptateur et décorateur
• Leçon 4 — Patterns comportementaux : observateur et stratégie
• Leçon 5 — Architecture en couches et architecture hexagonale : initiation
• Leçon 6 — Refactoring et gestion de la dette technique
• Leçon 7 — Projet : appliquer trois patterns à un cas réel
• Travaux dirigés (TD)
• Travaux pratiques (TP)
• Exercices corrigés supplémentaires
• QCM d'évaluation
• Résumé — les notions importantes
• Vidéos associées aux leçons
• Sources et références

Comment utiliser ce cours : lis chaque leçon dans l'ordre, refais les exemples à la main, tente chaque exercice AVANT de lire son corrigé, puis termine par le QCM pour vérifier ce que tu as retenu.

Introduction à la matière

En première et deuxième année, tu as appris à écrire des programmes qui fonctionnent : boucles, classes, bases de données, interfaces. En entreprise, on découvre vite une vérité moins confortable : un programme passe l'essentiel de sa vie à être modifié. On ajoute une fonctionnalité, on corrige un comportement, on change de fournisseur de paiement. Un code « qui marche » mais que personne n'ose toucher coûte plus cher qu'un code propre. Ce cours traite donc de la qualité de conception : comment organiser tes classes pour que le changement reste facile, local et sans peur.

Deux outils conceptuels vont nous guider. D'abord les principes SOLID, cinq règles de conception orientée objet qui, appliquées avec bon sens, évitent la plupart des enchevêtrements. Ensuite les design patterns (patrons de conception) : des solutions éprouvées à des problèmes qui reviennent sans cesse — créer des objets sans connaître leur classe exacte, brancher un composant étranger, notifier plusieurs modules d'un événement. Un pattern n'est pas une recette à copier aveuglément : c'est un vocabulaire partagé et un point de départ de conception, à adapter à ton contexte.

Tout le cours suit un fil rouge unique : BiblioIAM, l'application de gestion de la bibliothèque de l'institut (documents, emprunts, pénalités, facturation). Chaque leçon y améliore la même base de code, en Python, avec des exemples courts que tu peux recopier et exécuter. La dernière leçon combine trois patterns dans un mini-projet complet. Garde en tête la règle d'or du cours : on ne conçoit pas pour le plaisir de concevoir ; on conçoit pour le prochain changement, celui qu'on ne connaît pas

encore.

Objectifs d'apprentissage

À la fin de ce cours, tu seras capable de :

- Expliquer les cinq principes SOLID et repérer leur violation dans un code donné.
- Refactorer une classe qui mélange plusieurs responsabilités en classes cohésives.
- Mettre en œuvre les patrons Fabrique et Singleton et justifier leur emploi.
- Mettre en œuvre les patrons Adaptateur et Décorateur sur un code existant.
- Mettre en œuvre les patrons Observateur et Stratégie pour découpler un comportement.
- Comparer architecture en couches et architecture hexagonale et brancher un adaptateur sur un port.
- Planifier un refactoring prudent : petits pas, tests, et suivi de la dette technique.

Leçon 1 — Principes de conception : le sigle SOLID expliqué par l'exemple

SOLID est un sigle qui regroupe cinq principes de conception orientée objet, popularisés par Robert C. Martin. S — Responsabilité unique (Single Responsibility) : une classe ne doit avoir qu'une seule raison de changer, c'est-à-dire un seul métier. O — Ouvert/fermé (Open/Closed) : on doit pouvoir étendre un comportement en ajoutant du code, sans modifier le code existant qui fonctionne. L — Substitution de Liskov : toute sous-classe doit pouvoir remplacer sa classe mère partout où celle-ci est utilisée, sans changer le comportement attendu. I — Ségrégation des interfaces : mieux vaut plusieurs petites interfaces ciblées qu'une énorme interface que la moitié des clients n'utilise qu'à moitié. D — Inversion des dépendances : les modules de haut niveau (les règles métier) ne doivent pas dépendre des détails techniques (base de données, fichiers), mais les deux doivent dépendre d'abstractions.

Ces principes se renforcent mutuellement. Si chaque classe a une seule responsabilité (S), elle est plus petite, plus facile à tester, et il y a plus de chances qu'on puisse l'étendre sans la rouvrir (O). Si les composants dépendent d'interfaces plutôt que de classes concrètes (D), on peut substituer une implémentation par une autre (L) et les interfaces restent petites si on les découpe par client (I). À l'inverse, une seule « classe monstre » qui calcule, affiche, envoie des e-mails et écrit dans la base viole les cinq principes à la fois : le moindre changement y risque de tout casser, et personne ne sait par où commencer pour la tester.

Attention cependant : SOLID n'est pas un dogme. Appliqué aveuglément, il produit des forêts de micro-classes et d'interfaces pour des programmes de cinquante lignes. La bonne question n'est pas « est-ce que mon code est SOLID ? » mais « où le changement est-il probable, et mon découpage le rend-il facile ? ». Dans notre fil rouge BiblioIAM, la classe Facture du magasin de la bibliothèque va nous servir de cobaye : elle calcule un total, l'imprime et envoie un e-mail — trois métiers, donc trois raisons de changer. L'exemple qui suit la découpe en trois classes dont chacune peut évoluer sans déranger les autres.

L'essentiel
SOLID = cinq principes complémentaires : S responsabilité unique, O ouvert à l'extension / fermé à la modification, L substitution de Liskov, I petites interfaces séparées, D dépendre des abstractions. Le but n'est pas la pureté théorique, mais un code où le changement reste local, sûr et testable.

Exemple — Une classe Facture qui viole S, puis sa version refactorée (sortie réelle vérifiée)

```
# AVANT : une seule classe fait tout (calcul, affichage, envoi)
class Facture:
def __init__(self, client, montant):
self.client = client
self.montant = montant
def total(self):
return self.montant * 1.18 # TVA de 18 %
def imprimer(self):
print("Facture de", self.client, ":", self.total(), "FCFA")
def envoyer_par_email(self):
print("Email envoyé à", self.client)

# APRÈS : une responsabilité par classe (principe S de SOLID)
class Facture:
def __init__(self, client, montant):
self.client = client
self.montant = montant
def total(self):
return self.montant * 1.18

class ImprimanteFacture:
def imprimer(self, facture):
print("Facture de", facture.client, ":", facture.total(), "FCFA")

class EnvoiEmail:
def envoyer(self, facture):
print("Email envoyé à", facture.client,
"— total :", facture.total(), "FCFA")

f = Facture("Librairie du Fleuve", 50000)
ImprimanteFacture().imprimer(f)
EnvoiEmail().envoyer(f)
```

À retenir

• S : une classe, une raison de changer. Si tu décris une classe avec des « et » partout, elle fait probablement plusieurs métiers.

• O, L, I, D parlent tous de la même idée sous des angles différents : dépendre de contrats (interfaces) plutôt que de détails concrets.

• SOLID se juge sur le changement réel : vise les endroits du code qui évoluent souvent, ne surdécoupe pas le reste.

Exercice

Voici une classe imaginée pour BiblioIAM : GestionnaireMembre — elle inscrit les membres, vérifie leurs emprunts, imprime les cartes de lecteur, envoie les rappels SMS et archive les anciens membres dans un fichier. 1) Combien de responsabilités distingues-tu ? 2) Propose un découpage en classes, en nommant chacune par son métier. 3) Pour chaque nouvelle classe, donne une raison plausible de changement qui ne concernerait qu'elle.

✔ Corrigé
1) On peut distinguer cinq responsabilités : le registre des membres (inscription, modification), le contrôle des emprunts (droits et quotas), l'impression des cartes, la communication par SMS, et l'archivage. 2) Découpage possible : RegistreMembres (inscrire, rechercher, modifier) ; ControleEmprunts (peut_emprunter, emprunts_en_cours) ; ImpressionCarte (imprimer la carte d'un membre) ; NotificationSMS (envoyer un rappel à un membre) ; ArchivageMembres (écrire les anciens membres dans un fichier). 3) Exemples de raisons de changement isolées : le format de la carte change → seule ImpressionCarte bouge ; l'opérateur SMS change → seul NotificationSMS ; la durée de conservation des archives devient légale → seul ArchivageMembres. La classe d'origine, elle, aurait dû être modifiée pour les cinq raisons : c'est exactement ce que le S de SOLID veut éviter.

Leçon 2 — Patterns de création : fabrique et singleton

Les patrons de création répondent à une question simple : qui décide quel objet construire, et où ce choix vit-il dans le code ? Quand le mot-clé de création d'objet est éparpillé partout, le jour où une règle change (un nouveau type de document, un paramétrage différent), il faut traquer chaque site de création. Le patron Fabrique (Factory) centralise cette décision : le code client demande « fabrique-moi un document de code R » et reçoit un objet prêt à l'emploi, sans connaître la classe exacte. Le client ne dépend plus que du contrat commun des objets fabriqués — ici, le fait qu'ils savent tous répondre à duree_pret().

Le patron Singleton répond au problème inverse : certains objets doivent exister en un seul exemplaire partagé — une configuration, un journal de bord, un pool de connexions. Le Singleton garantit qu'une classe n'a qu'une instance et fournit un point d'accès unique à cette instance. En Python, une manière simple de l'implémenter est de contrôler la création dans la méthode __new__ : la première demande construit l'objet, les suivantes renvoient le même. L'exemple ci-dessous le montre : deux demandes de Configuration renvoient exactement le même objet (l'identité est vérifiée par le programme lui-même).

Ces deux patrons ont leurs zones de vigilance. La fabrique devient précieuse dès qu'il y a plusieurs types à créer ou des règles de choix ; pour un seul type simple, une fonction ordinaire ou même une instanciation directe suffit — ne fabrique pas d'usine pour visser une ampoule. Le singleton, lui, est un état global déguisé : il rend les tests plus difficiles (l'objet partagé garde la mémoire des tests précédents) et peut cacher des dépendances. Réserve-le aux objets vraiment uniques par nature (configuration, journal) et évite d'en faire un fourre-tout accessible de partout.

L'essentiel
Fabrique : centraliser la création d'objets derrière une fonction ou une classe, pour que le client ignore la classe concrète créée. Singleton : garantir une instance unique partagée, utile pour une configuration ou un journal, mais à manier avec prudence car c'est un état global.

Exemple — Fabriquer des documents de bibliothèque et partager une configuration unique

```
# Singleton : une seule configuration partagée dans toute l'application
class Configuration:
_instance = None
def __new__(cls):
if cls._instance is None:
cls._instance = super().__new__(cls)
cls._instance.taux_tva = 0.18
cls._instance.devise = "FCFA"
return cls._instance

# Fabrique : crée le bon type de document selon un code simple
class Livre:
def __init__(self, titre): self.titre = titre
def duree_pret(self): return 21
class Revue:
def __init__(self, titre): self.titre = titre
def duree_pret(self): return 7
class DVD:
def __init__(self, titre): self.titre = titre
def duree_pret(self): return 3

def fabriquer_document(code, titre):
if code == "L": return Livre(titre)
if code == "R": return Revue(titre)
if code == "D": return DVD(titre)
raise ValueError("Code inconnu : " + code)

c1 = Configuration()
c2 = Configuration()
print("Une seule configuration partagée ?", c1 is c2)
doc = fabriquer_document("R", "Sciences et Avenir")
print(doc.titre, "-> prêt de", doc.duree_pret(), "jours")
```

À retenir

• Fabrique = un seul endroit décide « quel objet pour quelle demande » ; le reste du code ne connaît que le contrat commun.

• Singleton = instance unique garantie + point d'accès partagé ; en Python, on peut le contrôler dans __new__.

• Un singleton est un état global : parfait pour une configuration ou un journal, dangereux comme variable fourre-tout partagée par tout le programme.

Exercice

BiblioIAM doit créer des objets de notification selon le canal choisi par le membre : « sms » → NotificationSMS (méthode envoyer(message) qui affiche « SMS : … »), « email » → NotificationEmail (affiche « Email : … »), « aucun » → NotificationSilencieuse (n'affiche rien). Écris la fonction fabriquer_notification(canal) qui renvoie le bon objet et lève ValueError pour un canal inconnu, puis un court programme qui fabrique la notification « sms » et envoie le message « Votre livre est disponible ».

✔ Corrigé
class NotificationSMS : def envoyer(self, message) : print("SMS :", message). class NotificationEmail : def envoyer(self, message) : print("Email :", message). class NotificationSilencieuse : def envoyer(self, message) : pass. def fabriquer_notification(canal) : si canal == "sms" → NotificationSMS() ; si "email" → NotificationEmail() ; si "aucun" → NotificationSilencieuse() ; sinon ValueError("Canal inconnu : " + canal). Programme : n = fabriquer_notification("sms") ; n.envoyer("Votre livre est disponible") — sortie réelle : « SMS : Votre livre est disponible ». Remarque le bénéfice : le code qui envoie ne connaît ni les classes ni le choix du canal ; ajouter un canal « WhatsApp » plus tard consiste à ajouter une classe et une branche dans la fabrique, sans toucher aux appelants (c'est le O de SOLID en action).

Leçon 3 — Patterns structurels : adaptateur et décorateur

Les patrons structurels s'occupent de la manière dont les objets s'assemblent. Le plus fréquent dans la vraie vie est l'Adaptateur (Adapter) : tu disposes d'un composant correct mais dont l'interface ne correspond pas à ce qu'attend ton application — unités différentes, noms de méthodes différents, format de données différent. Plutôt que de modifier le composant (parfois impossible : c'est une bibliothèque externe ou du vieux code validé), tu écris une petite classe traductrice qui présente l'interface attendue et délègue au composant en convertissant les appels. Dans l'exemple, l'ancien terminal de paiement raisonne en centimes avec une méthode debit ; notre application raisonne en FCFA avec une méthode payer ; l'adaptateur fait la conversion dans les deux sens du contrat.

Le Décorateur (Decorator) répond à un autre besoin : ajouter des comportements à un objet sans modifier sa classe et sans créer une explosion de sous-classes. L'idée est d'envelopper l'objet dans un autre objet de même interface, qui ajoute sa touche puis délègue à l'objet enveloppé. Les décorateurs s'empilent comme des oignons : une location de DVD, plus une assurance, plus une livraison. Sans ce patron, il faudrait des classes LocationAvecAssurance, LocationAvecLivraison, LocationAvecAssuranceEtLivraison… et encore une pour chaque nouvelle option. Avec lui, chaque option est une petite classe indépendante, combinable à volonté.

Le point commun de ces deux patrons est la composition : on assemble des objets plutôt que d'hériter à tout-va. L'héritage fige les choix à l'écriture du code ; la composition les rend modifiables à l'exécution — on peut brancher un autre terminal derrière le même adaptateur demain, ou décider au moment de la commande quelles options décorent la location. Retiens le critère de choix : adaptateur quand il faut rendre compatibles deux interfaces qui ne se connaissent pas ; décorateur quand il

faut enrichir un comportement par couches combinables.

L'essentiel
Adaptateur : une classe traductrice qui branche un composant existant sur l'interface attendue, sans modifier ni l'un ni l'autre. Décorateur : envelopper un objet dans des objets de même interface pour empiler des options, au lieu de multiplier les sous-classes.

Exemple — Brancher un ancien terminal de paiement et décorer une location d'options

```
# --- Adaptateur : brancher un ancien terminal de paiement ---
class AncienTerminal: # code existant, impossible à modifier
def debit(self, centimes):
return centimes <= 500000 # plafond : 5 000 FCFA

class AdaptateurTerminal: # traduit vers l'interface attendue
def __init__(self, terminal):
self.terminal = terminal
def payer(self, montant_fcfa):
ok = self.terminal.debit(montant_fcfa * 100) # FCFA -> centimes
return "Paiement accepté" if ok else "Paiement refusé"

# --- Décorateur : ajouter des options à un prix de base, sans le modifier ---
class LocationSimple:
def prix(self): return 2000
def description(self): return "Location du DVD"
class Option:
def __init__(self, base): self.base = base
class Assurance(Option):
def prix(self): return self.base.prix() + 500
def description(self): return self.base.description() + " + assurance"
class Livraison(Option):
def prix(self): return self.base.prix() + 1000
def description(self): return self.base.description() + " + livraison"

adapt = AdaptateurTerminal(AncienTerminal())
print(adapt.payer(3000), "|", adapt.payer(9000))
commande = Livraison(Assurance(LocationSimple()))
print(commande.description(), ":", commande.prix(), "FCFA")
```

À retenir

- Adaptateur = traduction d'interface : il protège ton code des changements d'un composant extérieur, et inversement.
- Décorateur = même interface, comportement enrichi, options empilables à l'exécution (base → options dans l'ordre choisi).
- Préfère la composition (assembler des objets) à l'héritage quand les combinaisons doivent rester ouvertes.

Exercice

Le nouveau lecteur de codes-barres de la bibliothèque fournit les informations d'un livre sous forme de dictionnaire {"isbn": ..., "titre": ...}, mais l'ancienne imprimante d'étiquettes n'accepte qu'une chaîne de texte au format « ISBN | Titre ». Écris un adaptateur qui permet d'imprimer l'étiquette du livre {"isbn": "978-001", "titre": "Réseaux"} avec l'ancienne imprimante, puis donne la sortie exacte attendue.

✔ Corrigé
class AncienneImprimante : def imprimer(self, texte) : return "Étiquette : " + texte. class AdaptateurEtiquette : __init__ reçoit l'imprimante ; imprimer_etiquette(self, livre) construit texte = livre["isbn"] + " | " + livre["titre"] puis délègue : return self.imprimante.imprimer(texte). Programme : a = AdaptateurEtiquette(AncienneImprimante()) ; print(a.imprimer_etiquette({"isbn": "978-001", "titre": "Réseaux"})). Sortie réelle vérifiée par exécution : « Étiquette : 978-001 | Réseaux ». Ni l'imprimante ni le lecteur n'ont été modifiés : toute la connaissance de la traduction vit dans l'adaptateur, qui sera le seul à changer si le format d'étiquette évolue.

Leçon 4 — Patterns comportementaux : observateur et stratégie

Les patrons comportementaux organisent la manière dont les objets interagissent et se répartissent les algorithmes. Le patron Stratégie (Strategy) extrait un comportement variable dans une famille de petites classes interchangeables : au lieu d'un gros « si profil = étudiant alors... sinon si profil = premium alors... » au milieu du code, on confie le calcul à un objet stratégie que l'on peut remplacer. Dans BiblioIAM, la pénalité de retard vaut 100 FCFA par jour en tarif standard, 50 pour les étudiants, 0 pour les membres premium. La classe Emprunt ne connaît aucune de ces règles : elle reçoit une stratégie et lui demande le montant. Ajouter un tarif « enseignant » demain ne touchera pas Emprunt (principe O).

Le patron Observateur (Observer) résout le problème de la notification : quand un événement se produit — un livre réservé vient d'être rendu — plusieurs parties du système doivent réagir (prévenir les membres en attente, mettre à jour les statistiques, écrire au journal). Si la bibliothèque connaît et appelle chacune de ces parties, elle devient un nœud fragile qui change à chaque nouvel intéressé. Avec Observateur, on inverse la relation : les intéressés s'abonnent auprès de l'émetteur, et l'émetteur, au moment de l'événement, prévient tous ses abonnés via une méthode commune (notifier), sans savoir qui ils sont ni ce qu'ils feront de l'information.

Ces deux patrons sont les briques de presque tous les systèmes réactifs modernes : interfaces graphiques (un bouton notifie ses écouteurs), applications mobiles (un modèle notifie les vues), architectures événementielles. Leur coût est une certaine indirection : le fil d'exécution saute d'objet en objet, ce qui demande de bons noms et parfois un schéma UML pour s'y retrouver — tu vois le lien avec le cours de modélisation de L2. La règle pratique : utilise Stratégie dès qu'un même calcul existe en plusieurs variantes qu'il faut choisir à l'exécution ; utilise Observateur dès qu'un événement doit déclencher des réactions chez un nombre variable d'intéressés.

L'essentiel
Stratégie : encapsuler chaque variante d'un algorithme dans une classe interchangeable, choisie à l'exécution. Observateur : un émetteur notifie tous ses abonnés via un contrat commun, sans les connaître — l'abonnement remplace les appels en dur.

Exemple — Pénalités de retard interchangeables et retour de livre qui notifie les membres

```
# --- Stratégie : plusieurs façons interchangeables de calculer une pénalité ---
class PenaliteStandard:
def calculer(self, jours): return jours * 100
class PenaliteEtudiant:
def calculer(self, jours): return jours * 50
class PenalitePremium:
def calculer(self, jours): return 0

class Emprunt:
def __init__(self, strategie):
self.strategie = strategie # on peut la changer à tout moment
def montant_penalite(self, jours_retard):
return self.strategie.calculer(jours_retard)

# --- Observateur : prévenir les membres quand un livre redevient disponible ---
class Bibliotheque:
def __init__(self):
self.abonnes = []
def abonner(self, membre):
self.abonnes.append(membre)
def livre_rendu(self, titre):
for m in self.abonnes: # elle ignore qui sont les abonnés
m.notifier(titre)

class Membre:
def __init__(self, nom): self.nom = nom
def notifier(self, titre):
print(self.nom, ": le livre «", titre, "» est disponible !")

e = Emprunt(PenaliteEtudiant())
print("Pénalité étudiant, 4 jours de retard :", e.montant_penalite(4), "FCFA")
bib = Bibliotheque()
bib.abonner(Membre("Awa"))
bib.abonner(Membre("Moussa"))
bib.livre_rendu("Algorithmique")
```

À retenir

• Stratégie : si tu vois un gros aiguillage (si/sinon) qui choisit un calcul, c'est probablement une stratégie qui s'ignore.

• Observateur : l'émetteur ne connaît que le contrat « notifier » ; abonner ou désabonner un intéressé ne modifie pas l'émetteur.

• Ces deux patrons échangent de la simplicité d'écriture contre du découplage : justifiés dès que variantes ou abonnés changent souvent.

Exercice

BiblioIAM veut prévenir différemment les membres quand un livre réservé devient disponible : par SMS, par e-mail, ou ne rien faire pour les membres qui n'ont rien demandé. Sans écrire tout le code, décris comment le patron Observateur s'applique ici : qui est l'émetteur, qui sont les observateurs, quelle méthode commune doivent-ils offrir ? Puis explique ce qu'il faut changer (et ne pas changer) le jour où l'on ajoute un observateur « Statistiques » qui compte les retours.

✔ Corrigé
Émetteur : l'objet qui gère les retours de livres (par exemple le service de prêts), qui maintient la liste des abonnés et expose abonner(). Observateurs : un objet NotificationSMS, un objet NotificationEmail, et éventuellement un objet qui ne fait rien — chacun offre la même méthode, par exemple notifier(titre, membre) ou, plus simplement ici, une méthode qui reçoit l'événement « livre disponible » et agit selon son canal. Au retour du livre, l'émetteur parcourt ses abonnés et appelle notifier sur chacun, sans savoir quel canal se cache derrière. Ajout de « Statistiques » : on crée la classe StatistiquesRetours avec la méthode notifier (qui incrémente un compteur) et on l'abonne. On ne change RIEN à l'émetteur ni aux autres observateurs : c'est exactement le principe ouvert/fermé rendu possible par le contrat commun des observateurs.

Leçon 5 — Architecture en couches et architecture hexagonale : initiation

Les patrons organisent les objets ; l'architecture organise les grandes parties du logiciel. L'architecture en couches (layered architecture) découpe l'application en étages horizontaux : présentation (écrans, API), métier (règles de gestion), accès aux données (persistance). Chaque couche ne parle qu'à celle du dessous : un écran n'interroge jamais directement la base de données, il passe par le métier, qui seul connaît les règles (« un membre ne peut pas emprunter plus de cinq livres »). Ce découpage simple couvre la majorité des applications de gestion et il rend les responsabilités lisibles : une erreur de règle se cherche dans la couche métier, une erreur d'affichage dans la présentation.

L'architecture hexagonale (ou « ports et adaptateurs », proposée par Alistair Cockburn) pousse la logique plus loin : au centre, le cœur métier ; autour, des ports — des interfaces qui décrivent ce dont le métier a besoin (un dépôt de livres, un service d'envoi de messages) ou ce qu'il offre (ses cas d'utilisation). À l'extérieur, des adaptateurs concrétisent les ports : un dépôt en mémoire pour les tests, un dépôt fichier ou SQL en production, une interface console aujourd'hui, une interface web demain. Le bénéfice décisif : le métier ne dépend d'aucune technologie. Changer de base de données ou brancher une nouvelle interface ne touche pas une ligne du cœur — c'est le principe D de SOLID appliqué à l'échelle du système entier.

Faut-il toujours viser l'hexagonal ? Non : pour un petit outil jetable, trois couches simples, voire un seul fichier bien organisé, suffisent, et l'hexagonal ajouterait des interfaces pour rien. La bonne échelle se choisit selon la durée de vie et la variabilité

attendue : si l'on sait que le stockage ou les canaux de notification vont changer, les ports protègent l'investissement ; sinon, ils le retardent. L'exemple ci-dessous montre la mécanique complète en miniature : un port DepotLivres, deux adaptateurs interchangeables, et un service métier qui fonctionne à l'identique avec l'un ou l'autre.

L'essentiel
Couches : présentation → métier → données, chacune ne parlant qu'à la suivante. Hexagonale : le métier au centre, des ports (interfaces) autour, des adaptateurs technologiques branchés à l'extérieur — le cœur ne dépend d'aucune technologie, on change d'adaptateur sans toucher au métier.

Exemple — Un port, deux adaptateurs : le catalogue de livres ne change jamais

```
# Architecture hexagonale : le cœur métier ne dépend que d'un "port"
class DepotLivres: # PORT : ce qu'exige le métier
def ajouter(self, livre): raise NotImplementedError
def trouver(self, isbn): raise NotImplementedError

class DepotMemoire(DepotLivres): # Adaptateur 1 : en mémoire (tests)
def __init__(self): self.livres = {}
def ajouter(self, livre): self.livres[livre["isbn"]] = livre
def trouver(self, isbn): return self.livres.get(isbn)

class DepotFichier(DepotLivres): # Adaptateur 2 : fichier texte
def __init__(self, chemin): self.chemin = chemin
def ajouter(self, livre):
with open(self.chemin, "a", encoding="utf-8") as f:
f.write(livre["isbn"] + ";" + livre["titre"] + "\n")
def trouver(self, isbn):
try:
with open(self.chemin, encoding="utf-8") as f:
for ligne in f:
code, titre = ligne.strip().split(";")
if code == isbn:
return {"isbn": code, "titre": titre}
except FileNotFoundError:
return None
return None

class ServiceCatalogue: # le cœur métier, inchangé quel que
def __init__(self, depot): # soit l'adaptateur branché
self.depot = depot
def enregistrer(self, isbn, titre):
self.depot.ajouter({"isbn": isbn, "titre": titre})
return "Livre enregistré : " + titre

service = ServiceCatalogue(DepotMemoire())
print(service.enregistrer("978-001", "Réseaux et systèmes"))
print("Recherche :", service.depot.trouver("978-001"))
```

À retenir

• En couches, la règle d'or : jamais d'accès direct d'une couche à celle qui n'est pas sa voisine (l'écran ne parle pas à la base).

• En hexagonal, le métier définit SES ports ; ce sont les adaptateurs qui s'y conforment, pas l'inverse — d'où le nom « inversion des dépendances ».

• Choisis l'architecture selon la variabilité réelle : ports et adaptateurs là où la technologie changera, simplicité ailleurs.

Exercice

Dans le code de la leçon, on veut remplacer le dépôt en mémoire par le dépôt fichier, puis plus tard par une vraie base de données SQL. 1) Quelle est la seule ligne du programme principal à modifier pour passer de DepotMemoire à DepotFichier ? 2) Pour le passage à SQL, quelle classe faut-il créer et à quel contrat doit-elle se conformer ? 3) Explique pourquoi ServiceCatalogue n'a pas besoin de connaître le nom de l'adaptateur utilisé.

✔ Corrigé
1) Une seule ligne : service = ServiceCatalogue(DepotFichier("catalogue.txt")) au lieu de ServiceCatalogue(DepotMemoire()). Tout le reste du programme est inchangé. 2) Il faut créer une classe DepotSQL qui hérite du port DepotLivres et implémente ajouter(livre) et trouver(isbn) avec des requêtes SQL ; le contrat est exactement celui du port : mêmes noms de méthodes, mêmes types échangés (un dictionnaire livre). 3) ServiceCatalogue reçoit son dépôt de l'extérieur (injection par le constructeur) et ne l'utilise qu'à travers le port : il appelle ajouter et trouver, sans savoir si derrière il y a un dictionnaire, un fichier ou une base. C'est l'inversion des dépendances : le métier impose son besoin (le port), les technologies s'y adaptent. La sortie du programme démontre le bon fonctionnement avec l'adaptateur mémoire : « Livre enregistré : Réseaux et systèmes » puis le dictionnaire {'isbn': '978-001', 'titre': 'Réseaux et systèmes'} retrouvé par la recherche.

Leçon 6 — Refactoring et gestion de la dette technique

Le refactoring est l'art d'améliorer la structure interne d'un code sans changer son comportement externe. Le mot-clé est « sans changer » : les mêmes entrées doivent donner les mêmes sorties, sinon ce n'est plus un refactoring mais une modification fonctionnelle — les deux se font séparément, jamais dans le même geste. Les gestes de base sont petits et nommés : renommer une variable pour dire son rôle, extraire une fonction quand un bloc fait une chose identifiable, remplacer un nombre magique par une constante nommée, supprimer une duplication. Pris un à un, ces gestes semblent dérisoires ; répétés régulièrement, ils empêchent le code de pourrir.

La dette technique est la métaphore qui explique pourquoi l'on refactorise : comme une dette financière, un raccourci pris aujourd'hui (« on fera propre plus tard ») fait payer des intérêts à chaque modification future, sous forme de temps perdu et de bugs. Toute la dette n'est pas mauvaise : un raccourci délibéré, documenté et remboursé vite peut être un bon calcul (livrer avant une date de soutenance, par exemple). La dette dangereuse est celle qu'on accumule sans le savoir, jusqu'au jour

où la moindre évolution prend des semaines. La gérer, c'est la rendre visible : la noter là où elle vit, l'estimer, et rembourser en priorité celle qui se trouve dans les zones du code que l'on modifie souvent.

La méthode sûre tient en trois habitudes. D'abord, avant de refactorer une zone, s'assurer qu'on peut vérifier que le comportement ne change pas — idéalement des tests automatisés, sinon une liste de cas exécutés à la main avant et après (dans nos exemples, on compare les sorties chiffrées). Ensuite, avancer par petits pas vérifiables plutôt que par grande réécriture. Enfin, suivre la « règle du boy-scout » : laisser chaque fichier un peu plus propre qu'on ne l'a trouvé. L'exemple ci-dessous refactorise une fonction de facturation illisible : constantes nommées, une fonction par idée, et un défaut rendu visible — l'ancien code appliquait la remise fidélité à tout le monde sans le dire.

L'essentiel
Refactoring = changer la structure sans changer le comportement, par petits pas vérifiés. La dette technique se gère comme une dette d'argent : la rendre visible, payer les intérêts là où le code change souvent, rembourser par tranches — jamais par grande réécriture risquée.

Exemple — Refactorer une fonction de facturation : le résultat chiffré ne doit pas bouger

```
# AVANT : une seule fonction, nombres magiques, lignes presque copiées-collées
def facture(n, t, j):
r = 0
if t == 1:
r = n * 1000 + j * 100
if t == 2:
r = n * 2000 + j * 100
if t == 3:
r = n * 3000 + j * 100
r = r - (r * 10 / 100) # remise de 10 % appliquée à tout le monde (!)
return r

# APRÈS : constantes nommées, une fonction par idée, remise explicite
TARIFS = {1: 1000, 2: 2000, 3: 3000} # prix par catégorie de document
PENALITE_PAR_JOUR = 100
REMISE_FIDELITE = 0.10

def prix_location(categorie, nombre):
return TARIFS[categorie] * nombre

def prix_penalite(jours_retard):
return PENALITE_PAR_JOUR * jours_retard

def facture_refaite(nombre, categorie, jours_retard, fidele):
total = prix_location(categorie, nombre) + prix_penalite(jours_retard)
if fidele:
total = total * (1 - REMISE_FIDELITE)
return total

print("Ancien code :", facture(2, 2, 3))
print("Code refactoré :", facture_refaite(2, 2, 3, True))
```

À retenir

- Un refactoring se prouve : mêmes entrées, mêmes sorties (ici 3870.0 avant et après) — sinon, c'est une modification, à traiter séparément.
- Les trois gestes qui remboursent le plus : nommer les constantes, extraire les fonctions par idée, éliminer les duplications.
- La dette technique la plus coûteuse est celle des zones modifiées souvent : c'est là qu'on rembourse d'abord.

Exercice

La fonction d'origine appliquait la remise de 10 % à tous les clients, y compris les non-fidèles — le refactoring l'a transformée en paramètre explicite « fidele ». 1) Calcule à la main ce que facture(2, 2, 3) renvoyait et vérifie que facture_refaite(2, 2, 3, True) donne bien le même nombre. 2) Que renvoie facture_refaite(2, 2, 3, False) et pourquoi la différence avec l'ancien code est-elle ici un progrès de conception, alors que la règle du refactoring dit de ne pas changer le comportement ? 3) Cite deux autres gestes de refactoring visibles entre les deux versions.

✔ Corrigé
1) Ancien code : t = 2 → r = 2 × 2000 + 3 × 100 = 4300 ; remise : 4300 − 430 = 3870.0. Version refactorée avec fidele = True : prix_location = 4000, prix_penalite = 300, total 4300, × 0,90 = 3870.0. Identique : le refactoring est valide sur ce cas (vérifié aussi par l'exécution réelle des deux fonctions). 2) facture_refaite(2, 2, 3, False) renvoie 4300 (aucune remise). La différence est un progrès parce que le refactoring a mis en lumière un comportement caché et probablement faux (remise pour tout le monde) : la bonne pratique consiste à d'abord refactorer à comportement constant, puis à décider explicitement, avec le métier, de la correction — ici, rendre la remise conditionnelle et visible dans l'appel. La correction devient un choix assumé, pas un effet secondaire. 3) Autres gestes : remplacement des nombres magiques (1000/2000/3000, 100, 10 %) par des constantes nommées (TARIFS, PENALITE_PAR_JOUR, REMISE_FIDELITE) ; extraction de fonctions (prix_location, prix_penalite) qui supprime les trois blocs presque identiques ; renommage des paramètres (n, t, j → nombre, categorie, jours_retard).

Leçon 7 — Projet : appliquer trois patterns à un cas réel

Cette leçon finale assemble le cours entier sur le fil rouge BiblioIAM : le traitement des retours de prêt en retard. Le besoin métier est simple à énoncer : quand un membre rend un document en retard, la pénalité dépend de son profil (étudiant ou standard), et la caisse doit être prévenue du montant à encaisser. Trois difficultés de conception se cachent dans cette phrase : le calcul de la pénalité varie selon le profil et changera encore (nouvelles catégories de membres) ; le choix de la règle applicable ne doit pas être éparpillé dans le code ; plusieurs services veulent être informés du retour (aujourd'hui la caisse, demain les statistiques et le journal).

Chaque difficulté appelle le patron appris dans ce cours. Le calcul variable devient une Stratégie : PenStandard et PenEtudiant, interchangeables. Le choix de la stratégie est centralisé dans une Fabrique : fabriquer_penalite(profil) est le seul endroit du programme qui connaît la correspondance entre un profil et une classe. La notification devient un Observateur : RetourPret prévient tous les observateurs abonnés via retour_enregistre, sans connaître leur identité. Le résultat est un code où chaque futur changement a un domicile unique : nouveau profil → une classe de stratégie + une branche de fabrique ; nouvel intéressé → un abonnement ; nouveau tarif → une constante dans une stratégie.

Avant de coder un tel assemblage, un professionnel dessine : un petit diagramme de classes UML (celui de L2 suffit) montrant RetourPret, les stratégies, la fabrique et les observateurs, puis il code en gardant le diagramme à jour. Il rédige aussi la liste des cas de vérification avant de lancer : un étudiant avec 6 jours de retard doit produire 6 × 50 = 300 FCFA de pénalité et un message à la caisse ; un profil standard avec 6 jours produirait 600 FCFA. Si le programme affiche autre chose, on sait immédiatement quel patron — donc quel fichier — regarder. C'est cela, concevoir : rendre le comportement prévisible et les erreurs localisables.

L'essentiel
Un cas réel combine les patrons : Stratégie pour le calcul qui varie, Fabrique pour centraliser le choix de la variante, Observateur pour notifier un nombre ouvert de services. Chaque changement futur a ainsi un seul endroit où s'appliquer.

Exemple — Gestion des retours de prêt : Fabrique + Stratégie + Observateur en 32 lignes

```
# Projet : gestion des retours de prêt avec TROIS patterns combinés

# 1) STRATÉGIE : la pénalité de retard dépend du profil du membre
class PenStandard:
def calculer(self, jours): return jours * 100
class PenEtudiant:
def calculer(self, jours): return jours * 50

# 2) FABRIQUE : choisit la bonne stratégie selon le profil reçu
def fabriquer_penalite(profil):
if profil == "etudiant":
return PenEtudiant()
return PenStandard()

# 3) OBSERVATEUR : la caisse est prévenue à chaque retour en retard
class Caisse:
def retour_enregistre(self, membre, penalite):
print("Caisse : pénalité de", penalite, "FCFA pour", membre)

class RetourPret:
def __init__(self, observateurs):
self.observateurs = observateurs
def traiter(self, membre, profil, jours_retard):
strategie = fabriquer_penalite(profil) # fabrique
montant = strategie.calculer(jours_retard) # stratégie
for obs in self.observateurs: # observateurs
obs.retour_enregistre(membre, montant)
return montant

service = RetourPret([Caisse()])
total = service.traiter("Fatou", "etudiant", 6)
print("Total encaissé :", total, "FCFA")
```

À retenir

- Devant un besoin, décompose les difficultés : ce qui varie (stratégie), ce qui choisit (fabrique), ce qui réagit (observateur).
- Le bénéfice se mesure en changements : ajoute mentalement un profil « enseignant » et un observateur « Statistiques » — comptes les lignes touchées.
- Toujours annoncer les sorties attendues avant d'exécuter (ici : 300 FCFA pour Fatou) : c'est ton test, et il désigne le coupable en cas d'écart.

Exercice

On veut étendre le projet : les enseignants bénéficient d'une pénalité réduite à 25 FCFA par jour, et un nouvel observateur JournalBord doit écrire chaque retour dans le journal de la bibliothèque (afficher « Journal : retour de <membre>, <montant> FCFA »). 1) Quelles classes créer et quelle(s) ligne(s) existante(s) modifier ? 2) Écris le code des ajouts. 3) Donne la sortie complète attendue pour service.traiter("M. Diallo", "enseignant", 4) avec les deux observateurs abonnés (Caisse puis JournalBord).

✔ Corrigé
1) Créer la classe PenEnseignant (stratégie) et la classe JournalBord (observateur). Modifier une seule ligne existante : ajouter la branche « enseignant » dans fabriquer_penalite. L'abonnement se fait à la construction : RetourPret([Caisse(), JournalBord()]) — RetourPret, Caisse et les anciennes stratégies ne changent pas. 2) class PenEnseignant : def calculer(self, jours) : return jours * 25. Dans la fabrique : if profil == "enseignant" : return PenEnseignant() (placé avant le retour par défaut). class JournalBord : def retour_enregistre(self, membre, penalite) : print("Journal : retour de", membre, ",", penalite, "FCFA"). 3) Calcul : 4 jours × 25 = 100 FCFA. Sortie attendue, dans l'ordre des abonnements : « Caisse : pénalité de 100 FCFA pour M. Diallo » puis « Journal : retour de M. Diallo , 100 FCFA » (l'espace avant la virgule vient de print, c'est la sortie réelle de ce style d'affichage), et traiter renvoie 100. Le nombre de lignes touchées dans le code existant — une — est la preuve concrète que l'assemblage des trois patrons a bien isolé le changement.

Travaux dirigés (TD)

TD 1 — Traquer les violations de SOLID dans un extrait de code

On te donne la description d'une classe de BiblioIAM nommée ServiceBibliotheque. Elle contient les méthodes suivantes : emprunterLivre, rendreLivre, calculerPenalite, envoyerSMS, envoyerEmail, imprimerTicket, sauvegarderDansFichier, chargerDepuisFichier, afficherStatistiques. De plus, son constructeur crée lui-même ses objets d'accès au fichier et ses outils d'envoi. 1) Pour chaque principe SOLID, dis s'il est violé et par quelles méthodes. 2) Propose un redécoupage en classes et interfaces. 3) Explique comment l'inversion des dépendances change le constructeur.

✔ Corrigé du TD 1
1) S violé : au moins quatre métiers (gestion des prêts, communication, impression, persistance, statistiques). O fragilisé : ajouter un canal « WhatsApp » impose de rouvrir la classe. L : rien à dire ici faute d'héritage — un principe non applicable n'est pas un principe violé, il faut le signaler honnêtement. I menacé : les appelants de ServiceBibliotheque dépendent d'une interface géante alors qu'ils n'en utilisent qu'un bout. D violé : le constructeur fabrique ses dépendances concrètes (fichier, SMS) au lieu de les recevoir. 2) Redécoupage : ServicePrets (emprunter, rendre, calculer la pénalité — ou une Stratégie si les règles varient), Notificateur (interface) avec SmsNotificateur et EmailNotificateur, ImprimanteTicket, DepotPrets (interface) avec DepotPretsFichier, ServiceStatistiques. 3) Avec D, ServicePrets reçoit son DepotPrets et son Notificateur en paramètres du constructeur : il ne connaît que les interfaces. On peut alors le tester avec un dépôt en mémoire, puis brancher le vrai fichier en production sans toucher au métier — c'est la même mécanique que le port de la leçon 5.

TD 2 — Choisir le bon patron pour quatre situations

Pour chacune de ces situations rencontrées dans des projets réels, nomme le patron le plus adapté parmi ceux du cours (Fabrique, Singleton, Adaptateur, Décorateur, Observateur, Stratégie) et justifie en une ou deux phrases. a) L'application doit calculer des frais de port selon trois transporteurs aux règles différentes, choisis au moment de la commande. b) Une bibliothèque graphique externe dessine des boutons avec une méthode dessiner(x, y, texte), mais ton interface attend afficherBouton(libelle, position). c) La page d'accueil d'une boutique doit rafraîchir son compteur de panier dès qu'un article est ajouté, sans que le panier connaisse la page. d) Une application de retouche photo doit pouvoir empiler des filtres (luminosité, contraste, noir et blanc) dans n'importe quel ordre sur une image. e) Un seul fichier de configuration doit être lu et partagé par tous les modules du programme.

✔ Corrigé du TD 2
a) Stratégie : trois algorithmes de calcul interchangeables, choisis à l'exécution selon le transporteur ; le code de commande ne contient aucun aiguillage. b) Adaptateur : il faut traduire l'interface attendue (afficherBouton) vers celle de la bibliothèque externe (dessiner) sans modifier ni l'une ni l'autre. c) Observateur : le panier est l'émetteur, la page s'abonne ; le panier notifie « article ajouté » sans connaître l'existence de la page. d) Décorateur : chaque filtre enveloppe l'image (ou le filtre précédent) avec la même interface, ce qui rend les filtres empilables dans tout ordre — par héritage, il faudrait une classe par combinaison. e) Singleton : objet unique par nature (la configuration chargée une fois), avec un point d'accès partagé ; à condition de le garder en lecture seule pour ne pas en faire un état global mutable incontrôlé.

Travaux pratiques (TP)

TP 1 — Faire évoluer BiblioIAM : trois changements imposés

Pars du code du projet de la leçon 7 (retours de prêt). Réalise successivement, en exécutant le programme après chaque étape : 1) ajoute un profil « visiteur » dont la

pénalité est de 200 FCFA par jour, sans modifier la classe RetourPret ; 2) ajoute un observateur Statistiques qui compte le nombre de retours en retard traités et affiche le compteur à chaque retour ; 3) remplace la stratégie des étudiants par une règle plafonnée : 50 FCFA par jour, avec un maximum de 500 FCFA. Pour chaque étape, note les classes créées, les lignes modifiées dans le code existant et la sortie obtenue. Livrable : le fichier Python final et un tableau « changement / fichiers touchés / sortie vérifiée ».

✔ Attendus du TP 1
Étape 1 : créer PenVisiteur (jours * 200) et ajouter une branche « visiteur » dans fabriquer_penalite : une seule ligne modifiée, RetourPret intacte. Vérification : traiter("Awa", "visiteur", 3) → 600 FCFA, message caisse correspondant. Étape 2 : créer Statistiques avec un compteur initialisé à 0 et retour_enregistre qui l'incrémente puis affiche « Statistiques : X retour(s) en retard » ; l'abonner dans la liste des observateurs — aucune ligne de logique existante modifiée, seule la construction du service change. Étape 3 : modifier uniquement PenEtudiant : return min(jours * 50, 500). Vérification : traiter("Fatou", "etudiant", 6) reste à 300 FCFA ; traiter("Fatou", "etudiant", 15) donne 500 FCFA (plafond) au lieu de 750. Le tableau final doit montrer que chaque changement est resté dans le domicile prévu par les patrons : stratégie pour les règles, fabrique pour le choix, observateur pour les réactions — si une étape t'a obligé à toucher RetourPret, c'est le signe d'un couplage à corriger.

Exercices corrigés supplémentaires

Exercice 1

Singleton pratique : écris une classe Journal qui garantit une instance unique. Elle possède une liste « lignes » (créée une seule fois) et une méthode ecrire(message) qui ajoute le message à la liste. Démontre ensuite, dans un court programme, que deux variables obtenues par Journal() désignent le même objet et voient les mêmes lignes.

✔ Corrigé
class Journal : _instance = None ; def __new__(cls) : si cls._instance est None, créer l'instance (super().__new__(cls)) et initialiser lignes = [] ; renvoyer cls._instance ; def ecrire(self, message) : self.lignes.append(message). Programme : j1 = Journal() ; j2 = Journal() ; j1.ecrire("Emprunt du livre 978-001 par Fatou") ; print(j2.lignes) ; print("Même journal ?", j1 is j2). Sortie réelle vérifiée par exécution : ['Emprunt du livre 978-001 par Fatou'] puis « Même journal ? True ». Le message écrit via j1 est visible via j2 parce qu'il n'existe qu'un seul journal : c'est exactement la propriété recherchée pour un journal de bord ou une configuration — et la raison pour laquelle il faut éviter d'y stocker des données de travail ordinaires.

Exercice 2

Liskov en pratique : la classe Document de BiblioIAM possède une méthode duree_pret() et tous les documents peuvent être empruntés. Un collègue crée la sous-classe DocumentConsultationSurPlace qui hérite de Document, mais dont la méthode emprunter() lève une erreur « consultation sur place uniquement ». 1)

Explique quelle lettre de SOLID est violée et pourquoi cela cassera du code existant. 2) Propose une conception corrigée qui distingue proprement les documents empruntables des autres.

✔ Corrigé
1) C'est le L (substitution de Liskov) : le code existant qui reçoit un Document s'attend à pouvoir appeler emprunter() ; avec la sous-classe, le même code plante à l'exécution. Une sous-classe ne doit ni renforcer les préconditions (exiger plus) ni affaiblir les promesses de sa classe mère. 2) Correction : ne pas faire hériter le document non empruntable de Document empruntable. On peut introduire une classe de base Document (titre, référence, duree_consultation…) et deux branches : DocumentEmpruntable (avec emprunter et duree_pret) et DocumentSurPlace (avec consulter, sans emprunter). Autre option, complémentaire (principe I) : deux petites interfaces, Empruntable et Consultable, que chaque document implémente selon son cas. Le code de prêt ne manipule alors que des Empruntable : l'erreur devient impossible par construction au lieu d'être découverte en production.

Exercice 3

Refactoring guidé : voici une fonction qui calcule le prix d'un abonnement annuel à la bibliothèque : def prix(cat) : si cat == 1 : return 5000 ; si cat == 2 : return 5000 - 5000 * 0.2 ; si cat == 3 : return 5000 - 5000 * 0.5 (catégories : 1 = extérieur, 2 = étudiant −20 %, 3 = personnel −50 %). Refactorise-la avec un tarif de base nommé, un dictionnaire de réductions et des noms de paramètres explicites, puis donne les trois prix obtenus avant et après pour prouver que le comportement n'a pas changé.

✔ Corrigé
TARIF_BASE = 5000 ; REDUCTIONS = {"exterieur": 0.0, "etudiant": 0.20, "personnel": 0.50} ; def prix_abonnement(categorie) : retourne TARIF_BASE * (1 − REDUCTIONS[categorie]). Prix avant refactoring : catégorie 1 → 5000 ; catégorie 2 → 5000 − 1000 = 4000 ; catégorie 3 → 5000 − 2500 = 2500. Prix après : exterieur → 5000 × 1,0 = 5000 ; etudiant → 5000 × 0,80 = 4000 ; personnel → 5000 × 0,50 = 2500. Identiques : refactoring valide. Bonus de conception : les catégories deviennent des noms lisibles plutôt que des codes 1/2/3, et ajouter une catégorie « partenaire −30 % » consiste à ajouter une ligne dans REDUCTIONS, sans toucher au calcul (principe O).

QCM d'évaluation

Choisis la bonne réponse pour chaque question. Les corrigés détaillés se trouvent juste après le QCM : ne les regarde qu'après avoir répondu à tout.

1. Dans le sigle SOLID, la lettre O signifie…

   A) Objets fermés à toute modification définitive

   B) Ouvert à l'extension, fermé à la modification

   C) Organisation en couches obligatoires

2. La lettre L (substitution de Liskov) exige que…

A) Toute classe ait exactement cinq méthodes

B) Les noms de classes commencent par leur type

C) Une sous-classe puisse remplacer sa classe mère sans changer le comportement attendu

3. Le patron Fabrique sert principalement à…

A) Garantir une instance unique d'une classe

B) Centraliser la décision de création d'objets

C) Notifier des abonnés lors d'un événement

4. Lequel de ces usages est un bon candidat pour un Singleton ?

A) La liste des emprunts d'un membre

B) La configuration de l'application, chargée une fois

C) Chaque facture émise par la caisse

5. Tu dois brancher un composant externe dont l'interface (noms et unités) ne correspond pas à celle de ton application, sans modifier le composant. Le patron adapté est…

A) Adaptateur

B) Observateur

C) Stratégie

6. Pour empiler des options combinables à volonté sur un objet (assurance, livraison, emballage…), sans créer une sous-classe par combinaison, on utilise…

A) Fabrique

B) Décorateur

C) Singleton

7. Dans le patron Observateur, quand un événement survient, l'émetteur…

A) Doit connaître la classe exacte de chaque abonné

B) Appelle la méthode commune de notification de tous ses abonnés

C) Attend que les abonnés viennent le consulter un par un

8. Le patron Stratégie est indiqué lorsque…

A) Un même calcul existe en plusieurs variantes à choisir pendant l'exécution

B) Il faut empêcher la création de deux objets identiques

C) Il faut traduire les appels d'une vieille bibliothèque

9. Dans une architecture hexagonale, un « port » est…

A) Un connecteur réseau numéroté

B) Une interface par laquelle le cœur métier échange avec l'extérieur

C) Un fichier de configuration du serveur

10. Refactorer un code signifie…

A) Réécrire entièrement l'application dans un autre langage

B) Ajouter la fonctionnalité demandée en en profitant pour tout réorganiser

C) Améliorer sa structure interne sans changer son comportement observable

Corrigés du QCM

1. Réponse B — Open/Closed : on étend un comportement en ajoutant du code (une classe, une stratégie) plutôt qu'en modifiant le code existant qui fonctionne.

2. Réponse C — Si un code fonctionne avec la classe mère, il doit fonctionner de la même façon avec n'importe laquelle de ses sous-classes.

3. Réponse B — La fabrique est le seul endroit qui choisit quelle classe concrète construire ; les clients reçoivent un objet conforme au contrat commun.

4. Réponse B — Le Singleton convient aux objets uniques par nature, comme une configuration ou un journal partagé — pas à des objets métier créés en nombre.

5. Réponse A — L'Adaptateur traduit les appels entre l'interface attendue et celle du composant, qui restent tous deux inchangés.

6. Réponse B — Chaque option est un décorateur de même interface que l'objet de base ; on les enveloppe les uns dans les autres à l'exécution.

7. Réponse B — L'émetteur ne connaît que le contrat (la méthode notifier ou équivalent) : c'est ce qui permet d'ajouter des abonnés sans le modifier.

8. Réponse A — Chaque variante devient une classe de stratégie interchangeable ; l'objet utilisateur délègue le calcul à la stratégie reçue.

9. Réponse B — Le port décrit un besoin ou une offre du métier (stocker des livres, envoyer un message) ; les adaptateurs le concrétisent avec une technologie donnée.

10. Réponse C — Le refactoring se prouve par l'invariance : mêmes entrées, mêmes sorties. Tout changement de comportement est une modification séparée, à décider explicitement.

Résumé — les notions importantes

• SOLID donne cinq réflexes de conception : une responsabilité par classe (S), étendre sans modifier (O), substituer sans surprise (L), petites interfaces (I), dépendre des abstractions (D).

• Fabrique centralise la création d'objets ; Singleton garantit une instance unique — utile pour config et journal, dangereux comme état global fourre-tout.

• Adaptateur traduit entre deux interfaces incompatibles sans modifier ni l'une ni l'autre ; Décorateur empile des options sur un objet grâce à une interface commune.

• Stratégie rend un calcul interchangeable à l'exécution ; Observateur diffuse un événement à des abonnés inconnus de l'émetteur, via un contrat de notification.

• Architecture en couches pour la lisibilité, architecture hexagonale quand la technologie autour du métier doit pouvoir changer : ports au centre, adaptateurs à l'extérieur.

• Le refactoring améliore la structure sans toucher au comportement, par petits pas vérifiés (mêmes entrées, mêmes sorties).

• La dette technique se gère comme un budget : la rendre visible, et rembourser d'abord celle des zones que l'on modifie le plus souvent.

Vidéos associées aux leçons

Ces vidéos, en français, complètent les leçons. Liens vérifiés au moment de la publication de ce document.

Leçon 1 — Les Secrets des Design Patterns : 23 Solutions pour Améliorer et Optimiser Votre Code

Chaîne : Solid Code App · Langue : Français · Niveau : Intermédiaire

Le panorama complet des 23 patrons du « Gang of Four », classés en patrons de création, structurels et comportementaux : exactement les trois familles traversées par ce cours. Regarde-la en ouverture pour situer chaque patron étudié ensuite un par un.

https://www.youtube.com/watch?v=LmVCsM_Yg1M

Leçon 2 — Les patrons de conception en Java — l'histoire complète, de 1977 à aujourd'hui

Chaîne : lioncode · Langue : Français · Niveau : Intermédiaire

D'où viennent les patrons, puis leur traversée en code Java : Singleton, Fabrique, Adaptateur, Décorateur, Stratégie et Observateur y ont chacun leur chapitre. Un complément idéal des leçons 2 à 4, avec des exemples Java proches de ceux du cours INF-L2-01.

https://www.youtube.com/watch?v=w1cVuooJmEs

Leçon 4 — Le Design Pattern COMMAND en Java — Cours Complet (comment marche vraiment Ctrl+Z)

Chaîne : lioncode · Langue : Français · Niveau : Intermédiaire

Un patron comportemental de plus, cousin de Stratégie et d'Observateur : transformer chaque action en objet pour pouvoir l'annuler, la journaliser ou la mettre en file. Le prolongement naturel de la leçon 4, avec l'implémentation Java complète et la pile d'annulation déroulée pas à pas.

https://www.youtube.com/watch?v=Yfihp-TqErI

Leçon 7 — Progresser en conception avec les Design Patterns (Benjamin Yvernault & Mathilde Lorrain) #MiXiT2023

Chaîne : MiXiT · Langue : Français · Niveau : Intermédiaire à avancé

Deux développeurs expérimentés montrent, en programmation en direct, les patrons qu'ils utilisent réellement au quotidien dans leurs bases de code professionnelles. Parfait pour clore le cours : tu y verras le passage du code « qui marche » au code bien conçu, comme dans le projet de la leçon 7.

https://www.youtube.com/watch?v=K1wT13mnyBI

Sources et références

• Programme officiel Licence Informatique, Université IAM (Institut Africain de Management), Bamako — vérifier les intitulés d'UE auprès du secrétariat pédagogique.

• Gamma, Helm, Johnson & Vlissides — « Design Patterns : Elements of Reusable Object-Oriented Software » (Addison-Wesley, 1994) : le catalogue fondateur des 23 patrons, dit « du Gang of Four ».

• Robert C. Martin — « Clean Architecture » (Prentice Hall, 2017) et « Agile Software Development : Principles, Patterns, and Practices » : les formulations de référence des principes SOLID et des architectures en ports et adaptateurs.

• Martin Fowler — « Refactoring : Improving the Design of Existing Code » (Addison-Wesley, 2e éd. 2018) : le catalogue des gestes de refactoring et la notion de « mauvaises odeurs » de code.

• Alistair Cockburn — articles sur l'architecture hexagonale (« Hexagonal Architecture », 2005, en accès libre sur le site de l'auteur) : la présentation originale des ports et adaptateurs.

• Les vidéos listées dans la section « Vidéos associées aux leçons » de ce document (liens vérifiés à la publication).

Fin du cours — Bronbah Tech, Bibliothèque Universitaire IAM. Bon courage pour tes révisions !